

1. Dati del progetto

- **Nome e cognome:** [Samuele Quirino, Valerio Zago]
- **Classe:** [3Btlc]
- **Titolo del gioco:** Il Cacciatore di Comete
- **Data della consegna:** [Domenica 31 Maggio]

2. Idea del gioco

Il gioco è un **arcade di abilità** in 2D. Il giocatore controlla un'astronave alla base dello schermo e deve raccogliere oggetti benefici (comete d'oro) evitando quelli pericolosi (meteoriti grigi). L'obiettivo è totalizzare il punteggio più alto possibile. Abbiamo scelto questo genere perché permette di gestire bene le collisioni e la generazione casuale di oggetti, rendendo ogni partita diversa.

3. Analisi e progettazione

3.1 Regole del gioco

- **Movimento:** Solo orizzontale (Frecce Destra/Sinistra).
- **Punteggio:** Ogni cometa raccolta vale 1 punto.
- **Game Over:** Il gioco termina immediatamente se si tocca un meteorite.
- **Difficoltà:** Gli oggetti cadono a velocità crescente con l'aumento del livello.

3.2 Struttura del programma

Il programma segue il classico **Game Loop** (Ciclo di gioco):

1. **Inizializzazione:** Caricamento di Pygame e creazione degli oggetti.
2. **Gestione Eventi:** Lettura degli input della tastiera.
3. **Aggiornamento Logica:** Calcolo delle nuove posizioni e controllo delle collisioni tramite il metodo `rect.collidect()`.
4. **Rendering:** Disegno degli elementi sullo schermo e aggiornamento del display a 60 FPS.

3.3 Strutture dati importanti

- **Classe Giocatore:** Gestisce le coordinate x e y e l'area di collisione (Rect) dell'astronave.
- **Classe OggettoCadente:** Una classe versatile che, tramite un parametro `tipo`, definisce se l'oggetto è una cometa o un meteorite.
- **Lista oggetti_in_caduta:** Una lista dinamica che contiene tutti gli oggetti attivi sullo schermo. Gli oggetti vengono aggiunti in cima e rimossi quando escono dal fondo o vengono colpiti.

4. Sviluppo e uso dell'AI

4.1 Come ho lavorato

Abbiamo inserito su gemini una richiesta riguardo al fatto di creare una bozza di un gioco semplice ma divertente come quelli che funzionano senza connessione, e dopo aver adoperato un paio di modifiche manuali e supporti dati sempre da gemini il gioco era completo.

4.2 Come ho usato l'AI

Abbiamo usato l'AI per:

- **Generare lo scheletro del codice:** Abbiamo chiesto una struttura semplice con classi e funzioni.
- **Spiegazioni tecniche:** Abbiamo chiesto informazioni e chiarimenti su come funziona `pygame.display.flip()` rispetto a `update()`.
- **Correzione bug:** Ci ha aiutato a capire che dovevamo aggiornare il valore di `self.rect.x` ogni volta che cambiavo la posizione `self.x`, altrimenti il giocatore si muoveva ma la sua "Hitbox" restava immobile.

5. Test e risultati

5.1 Casi di test

Test	Cosa ho fatto	Cosa mi aspettavo	Risultato
Bordi schermo	Premuto Freccia Destra al limite	L'astronave deve fermarsi	Funziona
Collisione Cometa	Toccata una cometa d'oro	Punteggio aumenta di 1	Funziona
Collisione Meteorite	Toccato un meteorite grigio	Passaggio a schermata Game Over	Funziona
Modalità GOLD	Abbiamo iniziato con il primo livello	L'astronave doveva avere una invulnerabilità contro le meteore e ottenere due punti dalle comete	Funziona

5.2 Bug e problemi

Un problema riscontrato è stato il "tremolio" degli oggetti. Abbiamo risolto impostando un tetto fisso ai fotogrammi con `orologio.tick(60)`. Inizialmente le comete cadevano troppo velocemente; così abbiamo risolto calibrando la variabile `VELOCITA_OGGETTO`.

6. Conclusioni e riflessione personale

Questo progetto ci ha insegnato a gestire il tempo nel software (il timer per la generazione degli oggetti) e l'importanza della progettazione a classi. L'uso dell'AI è stato **intelligente**: non abbiamo solo copiato, ma abbiamo anche analizzato i commenti per capire la logica matematica dietro le collisioni.

7. Allegati

IL CODICE:

```
import pygame
```

```
import random
```

```
import time
```

```
import sys
```

```
# --- Costanti del gioco ---
```

```
LARGHEZZA_SCHERMO = 800
```

```
ALTEZZA_SCHERMO = 600
```

```
VELOCITA_GIOCATORE = 10
```

```
# Colori
```

```
NERO = (0, 0, 0)
```

```
BIANCO = (255, 255, 255)
```

BLU = (0, 191, 255)

ORO = (255, 215, 0)

GRIGIO = (169, 169, 169)

ROSSO = (255, 69, 0)

VIOLA_EG = (148, 0, 211)

Variabili globali

RECORD = 0

--- Inizializzazione ---

pygame.init()

**pygame.display.set_caption("Space Hunter: Odd Level
Power-Up")**

**schermo =
pygame.display.set_mode((LARGHEZZA_SCHERMO,
ALTEZZA_SCHERMO))**

orologio = pygame.time.Clock()

font_piccolo = pygame.font.SysFont("Verdana", 20)

**font_medio = pygame.font.SysFont("Verdana", 35,
bold=True)**

font_titolo = pygame.font.SysFont("Verdana", 60, bold=True)

--- Classi ---

class Giocatore:

def __init__(self):

self.larghezza = 50

self.altezza = 30

self.x = LARGHEZZA_SCHERMO // 2 - self.larghezza //
2

self.y = ALTEZZA_SCHERMO - 70

self.colore_base = BLU

self.colore = BLU

self.rect = pygame.Rect(self.x, self.y, self.larghezza,
self.altezza)

Gestione God Mode

self.god_mode = False

self.god_start_time = 0

self.god_duration = 10

self.livello_ultimo_powerup = 0

```
def attiva_god_mode(self):
```

```
    self.god_mode = True
```

```
    self.god_start_time = time.time()
```

```
    self.colore = ORO
```

```
def disattiva_god_mode(self):
```

```
    self.god_mode = False
```

```
    self.colore = self.colore_base
```

```
def muovi(self, tasti):
```

```
    if tasti[pygame.K_LEFT] and self.x > 0:
```

```
        self.x -= VELOCITA_GIOCATORE
```

```
    if tasti[pygame.K_RIGHT] and self.x <  
LARGHEZZA_SCHERMO - self.larghezza:
```

```
        self.x += VELOCITA_GIOCATORE
```

```
    self.rect.x = self.x
```

```
def disegna(self, superficie):
```

```
    pygame.draw.rect(superficie, self.colore, self.rect,  
border_radius=5)
```

```
    cabina_colore = BIANCO if not self.god_mode else  
VIOLA_EG
```

```
pygame.draw.rect(superficie, cabina_colore, (self.x + 15,  
self.y - 5, 20, 10), border_radius=3)
```

```
class OggettoCadente:
```

```
    def __init__(self, tipo, velocita_extra):
```

```
        self.tipo = tipo
```

```
        if tipo == "cometa":
```

```
            self.colore = ORO
```

```
            self.dim = random.randint(15, 25)
```

```
            self.velocita = random.uniform(4, 6) + velocita_extra
```

```
        else:
```

```
            self.colore = GRIGIO
```

```
            self.dim = random.randint(30, 45)
```

```
            self.velocita = random.uniform(5, 8) + velocita_extra
```

```
        self.x = random.randint(0, LARGHEZZA_SCHERMO -  
self.dim)
```

```
        self.y = -50
```

```
        self.rect = pygame.Rect(self.x, self.y, self.dim, self.dim)
```

```
    def update(self):
```

```
self.y += self.velocita
```

```
self.rect.y = self.y
```

```
def disegna(self, superficie):
```

```
    if self.tipo == "cometa":
```

```
        pygame.draw.circle(superficie, self.colore,  
(self.rect.centerx, self.rect.centery), self.dim // 2)
```

```
    else:
```

```
        pygame.draw.rect(superficie, self.colore, self.rect,  
border_radius=8)
```

```
# --- Funzioni di supporto ---
```

```
def genera_stelle():
```

```
    return [[random.randint(0, LARGHEZZA_SCHERMO),  
random.randint(0, ALTEZZA_SCHERMO)] for _ in  
range(120)]
```

```
def mostra_testo(testo, font_usato, colore, x, y, centro=False):
```

```
    img_testo = font_usato.render(testo, True, colore)
```

```
    rect = img_testo.get_rect()
```

```
    if centro:
```



```
rect.center = (x, y)
```

```
schermo.blit(img_testo, rect)
```

```
else:
```

```
schermo.blit(img_testo, (x, y))
```

```
def menu_iniziale():
```

```
stelle = genera_stelle()
```

```
while True:
```

```
schermo.fill(NERO)
```

```
for s in stelle: pygame.draw.circle(schermo, BIANCO,  
(s[0], s[1]), 1)
```

```
mostra_testo("SPACE HUNTER", font_titolo, ORO,  
LARGHEZZA_SCHERMO//2, 250, True)
```

```
mostra_testo("God Mode attiva ad ogni LIVELLO  
DISPARI!", font_piccolo, ORO,  
LARGHEZZA_SCHERMO//2, 350, True)
```

```
mostra_testo("Premi SPAZIO per iniziare", font_piccolo,  
BIANCO, LARGHEZZA_SCHERMO//2, 400, True)
```

```
pygame.display.flip()
```

```
for ev in pygame.event.get():
```

```
if ev.type == pygame.QUIT: pygame.quit(); sys.exit()
```

```
if ev.type == pygame.KEYDOWN and ev.key ==  
pygame.K_SPACE: return
```

--- Main Game ---

def gioco():

global RECORD

giocatore = Giocatore()

stelle = genera_stelle()

oggetti = []

punteggio = 0

livello = 1

velocita_bonus = 0

frequenza_spawn = 1.0

ultimo_spawn = time.time()

running = True

while running:

schermo.fill(NERO)

for s in stelle: pygame.draw.circle(schermo, (180,180,180),
(s[0], s[1]), 1)

1. Logica Livelli e Power-Up

nuovo_livello = (punteggio // 10) + 1

if nuovo_livello > livello:

livello = nuovo_livello

velocita_bonus += 1.3

frequenza_spawn *= 0.90

**giocatore.colore_base = (random.randint(50,200),
random.randint(50,200), 255)**

**if not giocatore.god_mode: giocatore.colore =
giocatore.colore_base**

**if livello % 2 != 0 and giocatore.livello_ultimo_powerup
!= livello:**

giocatore.attiva_god_mode()

giocatore.livello_ultimo_powerup = livello

**# FIX: Il calcolo del tempo ora avviene SOLO se la God
Mode è attiva**

if giocatore.god_mode:

tempo_passato = time.time() - giocatore.god_start_time

**tempo_rimasto = max(0, giocatore.god_duration -
tempo_passato)**

Disegna UI God Mode

**larghezza_barra = (tempo_rimasto /
giocatore.god_duration) * 200**

**pygame.draw.rect(schermo, ORO,
(LARGHEZZA_SCHERMO//2 - 100, 40, ball_width := max(0,
larghezza_barra), 10))**

**mostra_testo("GOD MODE", font_piccolo, ORO,
LARGHEZZA_SCHERMO//2 - 50, 15)**

if tempo_rimasto <= 0:

**giocatore.disattiv_god_mode =
giocatore.disattiva_god_mode()**

for ev in pygame.event.get():

if ev.type == pygame.QUIT: pygame.quit(); sys.exit()

tasti = pygame.key.get_pressed()

giocatore.muovi(tasti)

if time.time() - ultimo_spawn > frequenza_spawn:

**tipo = "cometa" if random.random() < 0.7 else
"meteorite"**

oggetti.append(OggettoCadente(tipo, velocita_bonus))

```
ultimo_spawn = time.time()
```

Ottimizzazione: Gestione logica e disegno in un unico ciclo sicuro

```
for obj in oggetti[:]:
```

```
    obj.update()
```

```
    if obj.rect.colliderect(giocatore.rect):
```

```
        if obj.tipo == "cometa":
```

```
            punteggio += 2 if giocatore.god_mode else 1
```

```
            oggetti.remove(obj)
```

```
        else:
```

```
            if giocatore.god_mode:
```

```
                oggetti.remove(obj)
```

```
            else:
```

```
                running = False
```

```
    elif obj.y > ALTEZZA_SCHERMO:
```

```
        oggetti.remove(obj)
```

```
    else:
```

```
        obj.disegna(schermo) # Disegna solo se esiste ancora
```

```
giocatore.disegna(schermo)
```

```
mostra_testo(f"SCORE: {punteggio}", font_piccolo,  
BIANCO, 20, 20)
```

```
mostra_testo(f"LEVEL: {livello}", font_piccolo, ORO,  
20, 50)
```

```
mostra_testo(f"RECORD: {RECORD}", font_piccolo,  
ROSSO, LARGHEZZA_SCHERMO - 150, 20)
```

```
pygame.display.flip()
```

```
orologio.tick(60)
```

```
if punteggio > RECORD: RECORD = punteggio
```

```
return punteggio # FIX: Restituiamo il punteggio invece di  
chiamare la schermata direttamente
```

```
def schermata_game_over(punteggio):
```

```
    # FIX: Questa funzione ora gestisce solo la schermata e  
    restituisce True se si vuole rigiocare
```

```
    while True:
```

```
        schermo.fill(NERO)
```

```
        mostra_testo("MISSIONE FALLITA", font_titolo,  
ROSSO, LARGHEZZA_SCHERMO//2, 200, True)
```

```
    mostra_testo(f"Punteggio Finale: {punteggio}",  
font_medio, BIANCO, LARGHEZZA_SCHERMO//2, 300,  
True)
```

```
    mostra_testo("Premi SPAZIO per rigiocare o ESC per  
uscire", font_piccolo, BIANCO,  
LARGHEZZA_SCHERMO//2, 400, True)
```

```
pygame.display.flip()
```

```
for ev in pygame.event.get():
```

```
    if ev.type == pygame.QUIT: pygame.quit(); sys.exit()
```

```
    if ev.type == pygame.KEYDOWN:
```

```
        if ev.key == pygame.K_SPACE:
```

```
            return True
```

```
        if ev.key == pygame.K_ESCAPE:
```

```
            pygame.quit(); sys.exit()
```

```
if __name__ == "__main__":
```

```
    menu_iniziale()
```

```
    # FIX: Loop principale del programma che evita la  
    ricorsione distruttiva
```

```
    giocando = True
```

```
    while giocando:
```

```
        punti_ottenuti = gioco()
```

```
        giocando = schermata_game_over(punti_ottenuti)
```

gli screenshot del gioco li alleghiamo su classroom con una cartella.